

Buses and Interconnects and Hex 8 Memory

COMS10015

Dr Tom Deakin

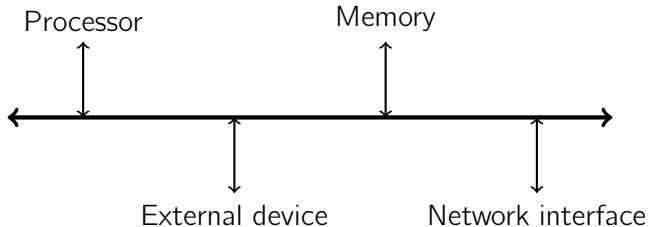
University of Bristol

Week 16

Buses

Definition: Bus

A *bus* is a communication system.



A bus takes data from one location to another.

Buses

Internal bus Connects components of a computer together.

- ▶ e.g. System bus connects processor, memory, I/O, ...
- ▶ System bus consists of data bus (for data), address bus (for destination information), and control bus (commands and status).

External bus Connect to other devices.

e.g. networked devices, discrete graphics card, ...

Buses

- ▶ Includes the medium (hardware/wires) and the protocol (software).
- ▶ Can be serial or parallel.
- ▶ Can be synchronous or asynchronous.
- ▶ Can be unidirectional (one-way flow) or bidirectional (send and receive).
- ▶ Often has multiple access points for parts to join the bus.
- ▶ Communication link might be constructed with separate control and data channels.
- ▶ Can design with different characteristics: bandwidth, latency, etc.

Serial and parallel buses

Serial bus

- ▶ Data message is sent one bit at a time.
- ▶ Connection is just a single channel.
- ▶ Use a SerDes to serialise parallel data, send on link, and deserialise.
- ▶ Often very high clock rate.
- ▶ Example: Universal Standard Bus (USB) and PCI Express.

Parallel bus

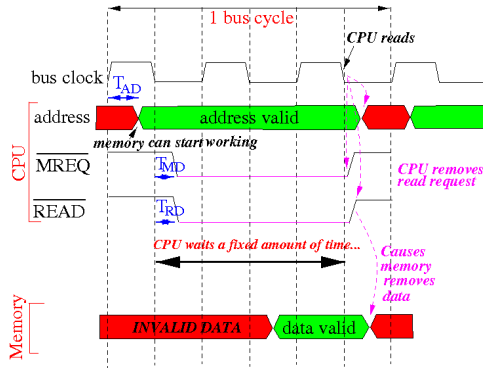
- ▶ Connection could have multiple channels (wires).
- ▶ Data message is split up and sent over all wires simultaneously.
- ▶ Example: Parallel ATA (used for hard disk drives before 2003 when superseded by SATA).
- ▶ Less commonly used today.

Synchronous buses

- ▶ Communication guided by clock signal.
- ▶ Clock signal shared between both ends of communication.
- ▶ Signals on control wires used to initiate protocol.
- ▶ Example: CPU reading from memory:

1. CPU send memory address.
2. Signals read request.
3. Waits fixed number of cycles.
4. CPU copies data off bus.
5. CPU removes read request signal.

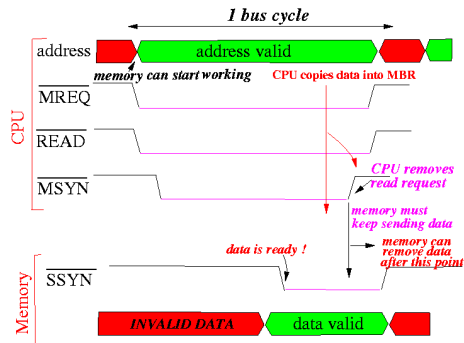
Picture from <http://www.mathcs.emory.edu/~cheung/Courses/355/Syllabus/5-bus/synch-bus.html>



Asynchronous buses

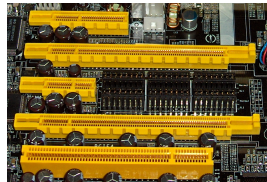
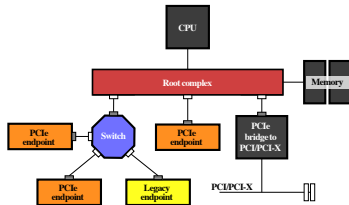
- ▶ Devices need to handshake before communication to ensure both are ready.
- ▶ Additional synchronisation signals used to share state.
- ▶ Example: CPU reading from memory:

1. CPU send memory address.
2. Signals read request.
3. CPU signals ready.
4. Waits (indefinitely) until memory signals read.
5. CPU copies data off bus.
6. CPU removes read request and ready signals.



Example: PCI Express

- ▶ Serial bus.
- ▶ Point-to-point topology.
- ▶ Separate serial lines to connect every device to root complex.
- ▶ Multiple lanes used to increase bandwidth: 1 to 32 lanes. PCIe x1 — PCIe x32.
- ▶ Each lane has two channels: one for send, one for receive.
 - ▶ Any two devices can share data in both directions simultaneously (dual-simplex).
 - ▶ Implemented with 2 wires per channel to prevent crosstalk.

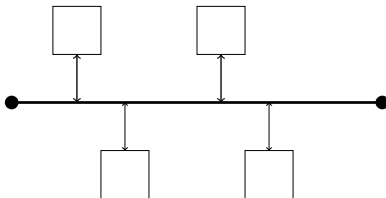


Pictures from https://en.wikipedia.org/wiki/PCI_Express#/media/File:Example_PCI_Express_Topology.svg and https://en.wikipedia.org/wiki/PCI_Express#/media/File:PCIExpress.jpg

Example: Ethernet

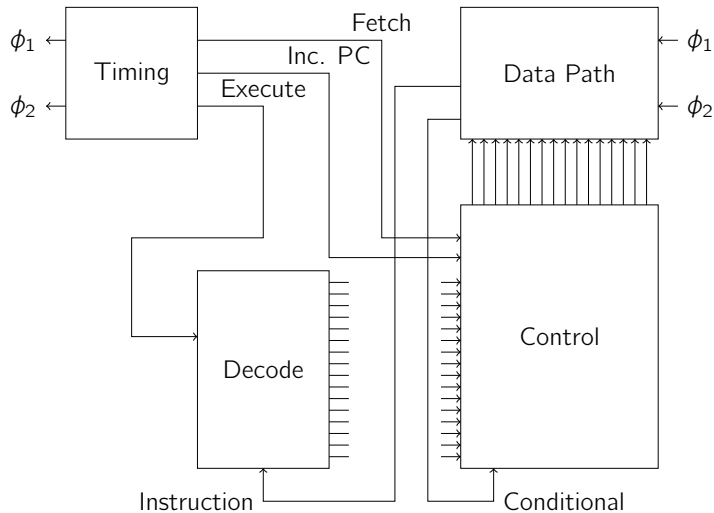
Ethernet commonly used to connect computers together.

- ▶ One cable connects all computers together.
- ▶ Modern topologies use switches to make more flexible.
- ▶ Operates with a bus protocol: each computer loads data and destination onto network.
- ▶ All packets sent to all computers: network card only interrupts at correct destination.



Hex 8 Micro-architecture: Memory

Hex 8 micro-architecture: Structure



Hex 8: ISA

Seen nine of the sixteen instructions:

ADD $areg \leftarrow areg + breg$: add registers and store in *areg*

SUB $areg \leftarrow areg - breg$: subtract registers and store in *areg*

LDAC $areg \leftarrow oreg$: load constant into *areg*

LDBC $breg \leftarrow oreg$: load constant into *breg*

LDAP $areg \leftarrow pc + oreg$: load address in program

BR $pc \leftarrow pc + oreg$: branch relative unconditional

BRZ if $areg = 0$ then $pc \leftarrow pc + oreg$: branch relative zero

BRN if $areg < 0$ then $pc \leftarrow pc + oreg$: branch relative negative

BRB $pc \leftarrow breg$: branch absolute

Hex 8: Data access

Hex 8: Data access

- ▶ Previously seen data access instructions which load operand into registers.
- ▶ Six more instructions to move data to/from main memory.
- ▶ ISA allows for 256 bytes of main memory.

Hex 8: Data access

LDAM $areg \leftarrow mem[oreg]$: load from memory address *oreg* into *areg*

LDBM $breg \leftarrow mem[oreg]$: load from memory address *oreg* into *breg*

STAM $mem[oreg] \leftarrow areg$: store *areg* in memory address *oreg*

- ▶ Load/store instructions to access main memory.
- ▶ Only data in *areg* can be stored to memory.
- ▶ Memory address is held in *oreg* — instruction operand.
- ▶ Example: LDAM 1110.
 - ▶ Set *oreg* to 1110 (14).
 - ▶ Load $mem[14]$ into *areg*.

Hex 8: Data access

LDAI $areg \leftarrow mem[areg + oreg]$: load from memory into *areg*

LDBI $breg \leftarrow mem[breg + oreg]$: load from memory into *breg*

STAI $mem[breg + oreg] \leftarrow areg$: store *areg* in memory

- ▶ Load/store instructions with offset to *oreg*.
- ▶ Offsets stored in either *areg* and *breg*.
- ▶ Allows for relative memory addressing — not possible with immediate values alone.

Hex 8: Relative data access

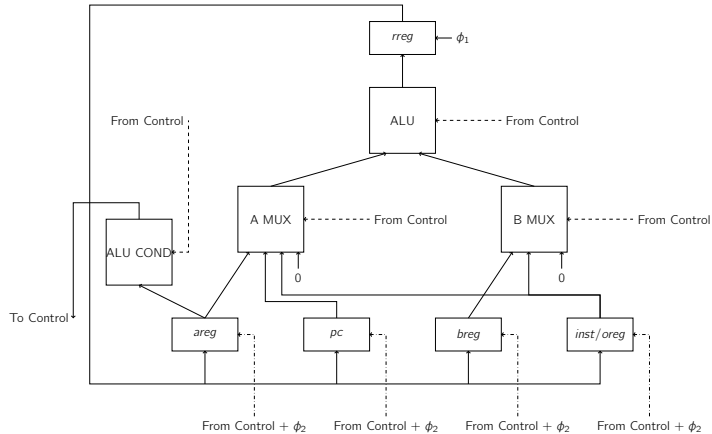
Relative memory addressing important for data structures.

Example: pointers/arrays: e.g. in C, `int A[8];`

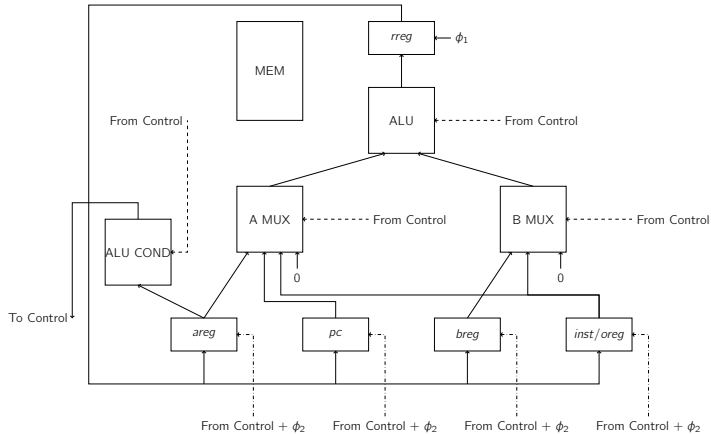
- ▶ Assume the array *A* is put into memory somewhere, and that location is stored at memory address 12 (more on how this happens next lecture).
- ▶ So *mem*[12] contains the start of the array *A*.
- ▶ Don't know the address where *A* starts, but we know that address is saved in memory, at address 12.
- ▶ Want to access *A*[4], an element of the array:
 1. LDAM 1100 to set *areg* to *mem*[12], the address of the start of *A*.
 2. LDAI 0100 to set *areg* to *mem*[*mem*[12] + 4], which is *A*[4].

Hex 8: Adding Memory to data path

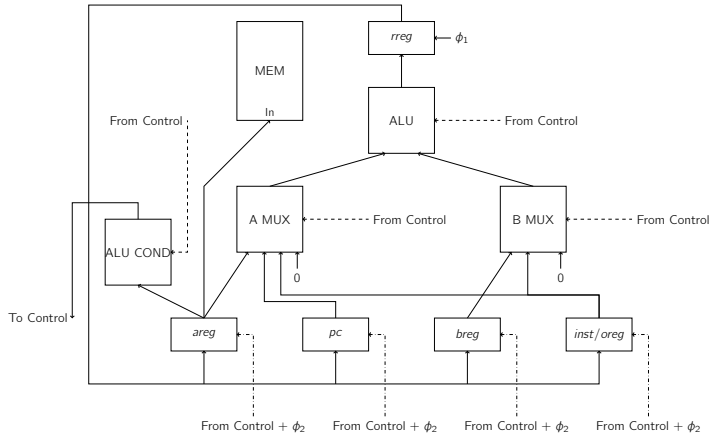
Hex 8: Adding Memory to data path



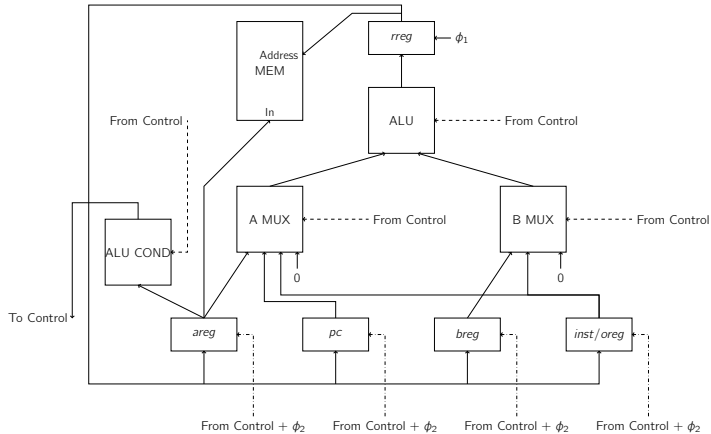
Hex 8: Adding Memory to data path



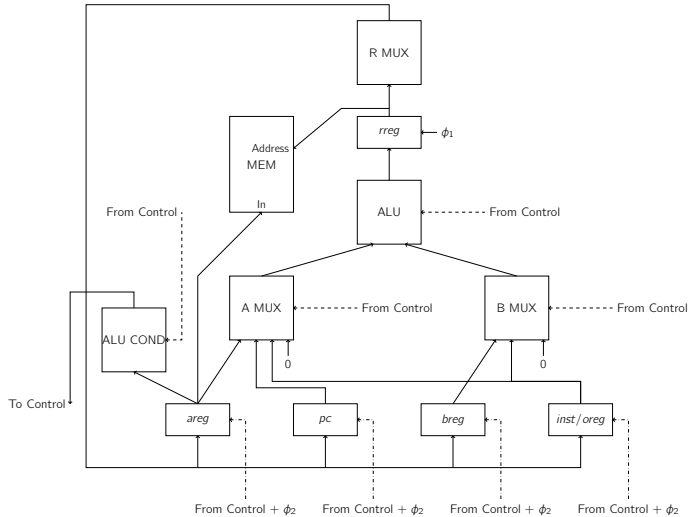
Hex 8: Adding Memory to data path



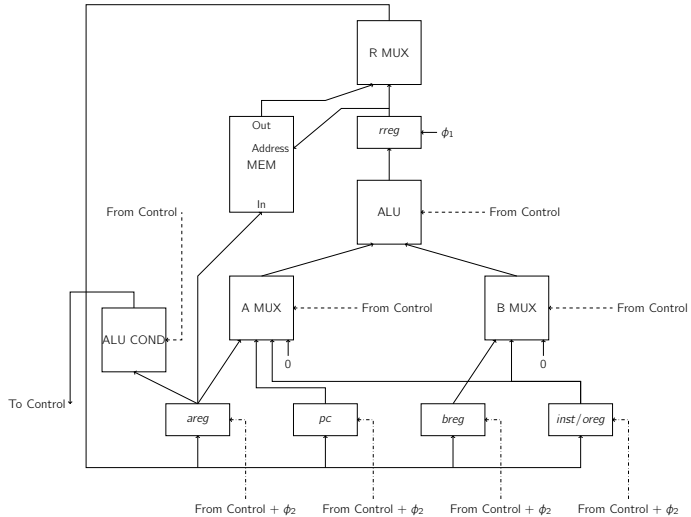
Hex 8: Adding Memory to data path



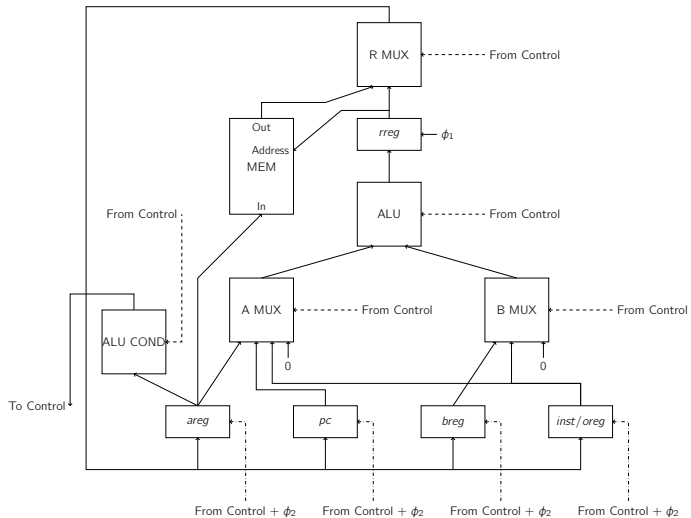
Hex 8: Adding Memory to data path



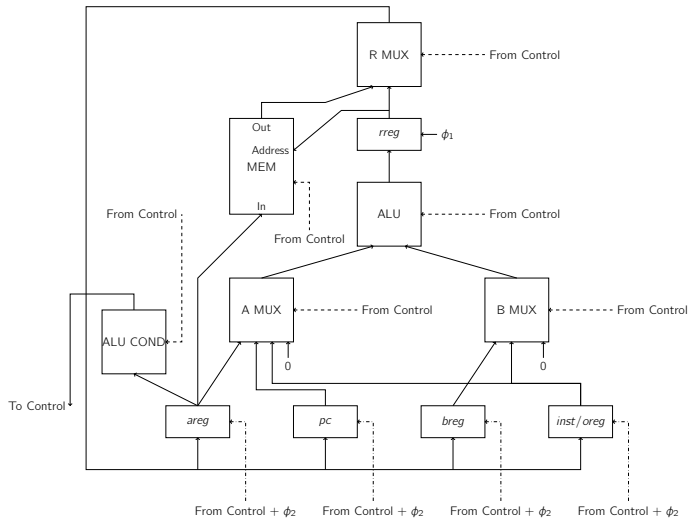
Hex 8: Adding Memory to data path



Hex 8: Adding Memory to data path



Hex 8: Adding Memory to data path



ModuleSim: Memory

In ModuleSim, the NRAM module provides memory.

- ▶ It's much larger than we need.
- ▶ The Hex 8 will only be able to access first 256 8-bit words.
- ▶ Only need to use two address inputs: A (least-significant 4 bits) and B (most-significant 4 bits).
- ▶ Control signals:
 - Write *1*1
 - Read *0**
- ▶ Safety switch to enable writes.

Instruction Fetch

Instruction fetch

Now we have memory, can implement more of the control logic.

- ▶ Recall Hex 8 execution cycle: Fetch, Increment PC, Execute.
- ▶ Timing unit generates these signals, along with the two-phase clock.
- ▶ On Fetch signal, load $mem[pc]$ and store in *inst/oreg* register.
- ▶ *inst* register is populated with instruction opcode.
- ▶ *oreg* register is populated with instruction operand.
- ▶ These registers then ready to be decoded and executed on Execute signal.

Prefix instruction

Hex 8: Operands

- ▶ Hex 8 instructions are 8-bits.
 - ▶ 4-bit opcode plus 4-bit operand.
 - ▶ *oreg* stores this operand.
 - ▶ Many instructions use *oreg*:
 - ▶ memory address, value to set registers, branch offset, ...
 - ▶ But instruction has only 4-bit operand, but sometimes want 8-bit operand.
 - ▶ *oreg* needs to be an 8-bit register.
 - ▶ Store opcode in own 4-bit register *inst*.
- Hex 8 ISA has a special **prefix** instruction to help construct 8-bit operands.

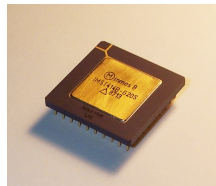


Image of Transputer from
Wikipedia CC-BY-2.5.

Hex 8: prefix instruction

PFIX concatenates 4-bit immediate operand with the 4-bit operand of the *next* instruction

The operand of the PFIX becomes the 4 most-significant (high) bits in the *oreg*.

► We denote this $oreg[4 : 7]$.

The operand of the next instruction becomes the 4 least-significant (low) bits in the *oreg*.

► We denote this $oreg[0 : 3]$.

Hex 8: prefix example

Example, want to load $mem[159]$.

- ▶ 159 in binary is 10011111.
- ▶ Need 8-bit operand to represent this number.
- ▶ Goal: need 8-bit *oreg* to be 10011111 when load instruction executed.
 1. PFIX 1001, prefix instruction with immediate value of the 4 high bits of address.
 - ▶ When PFIX is fetched, the operand 1001 is stored in the 4 low bits of *oreg*: $oreg[0 : 3]$.
 - ▶ When PFIX executed, moves low bits into high bits of *oreg*: $oreg[4 : 7] \leftarrow oreg[0 : 3]$.
 2. LDAM 1111, load memory with immediate value of the 4 low bits of address.
 - ▶ When LDAM is fetched, the operand 1111 is stored in $oreg[0 : 3]$.
 - ▶ $oreg[0 : 7]$ is now 10011111.
 - ▶ When LDAM is executed, this loads $mem[oreg]$ and stores in *areg*.

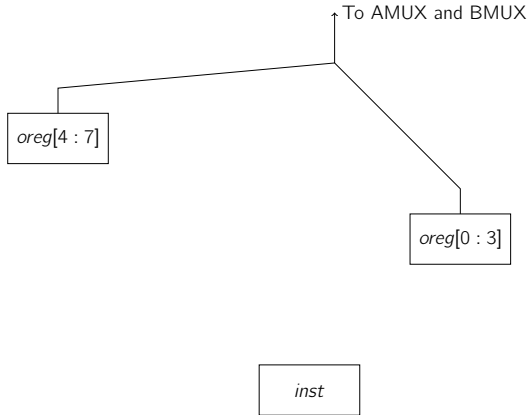
Hex 8: Update to inst/oreg registers

oreg[4 : 7]

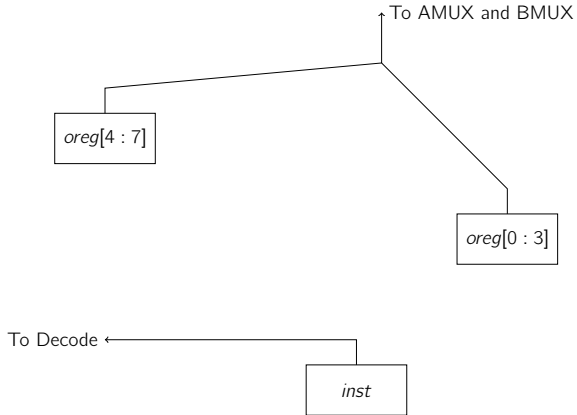
oreg[0 : 3]

inst

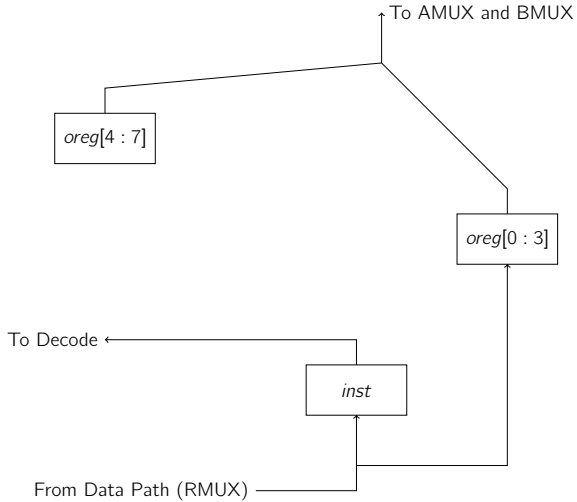
Hex 8: Update to inst/oreg registers



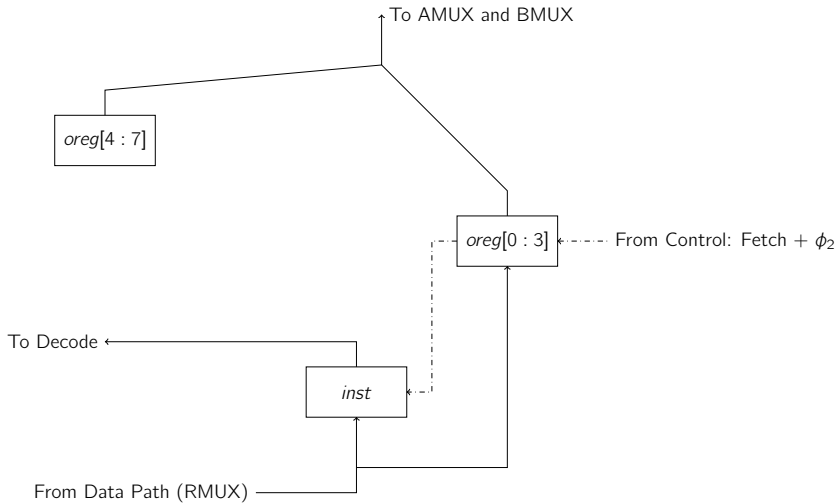
Hex 8: Update to inst/oreg registers



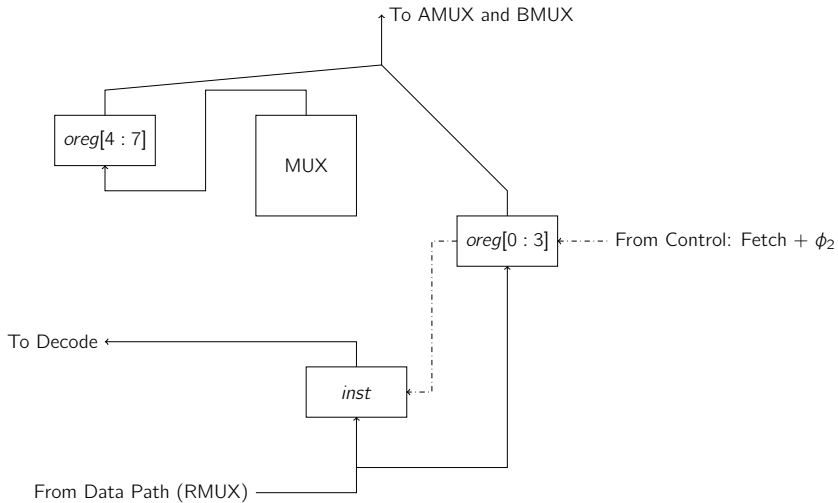
Hex 8: Update to inst/oreg registers



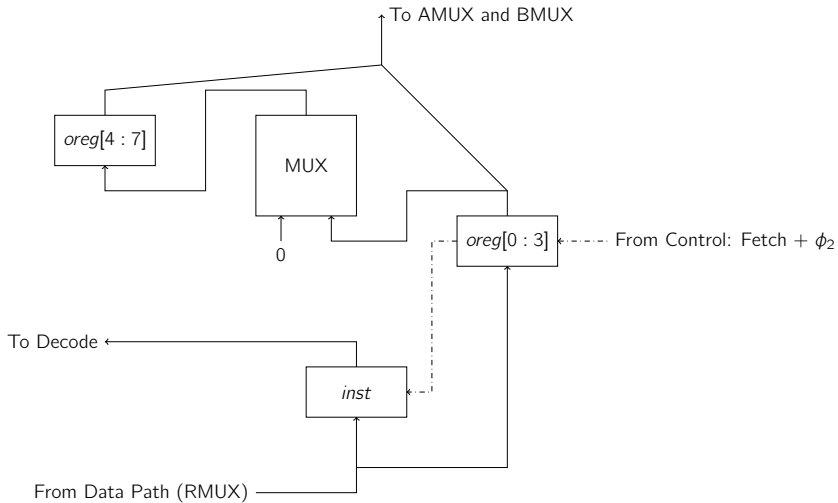
Hex 8: Update to inst/oreg registers



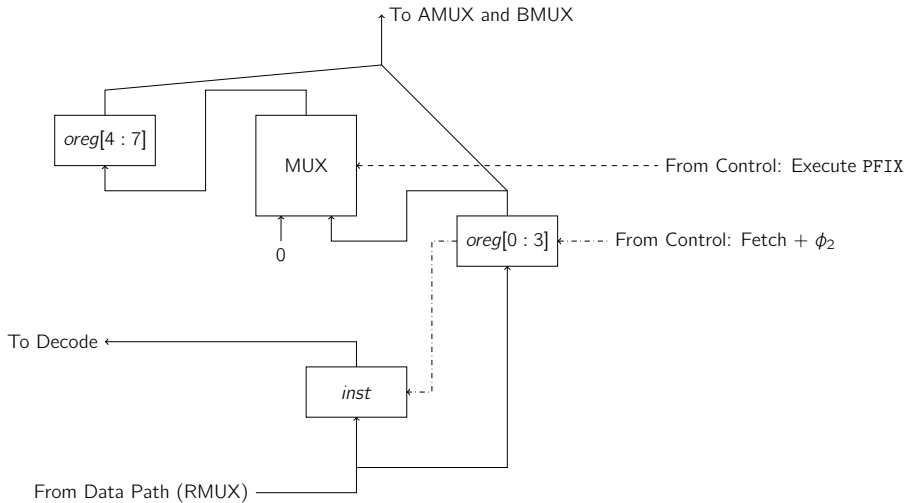
Hex 8: Update to inst/oreg registers



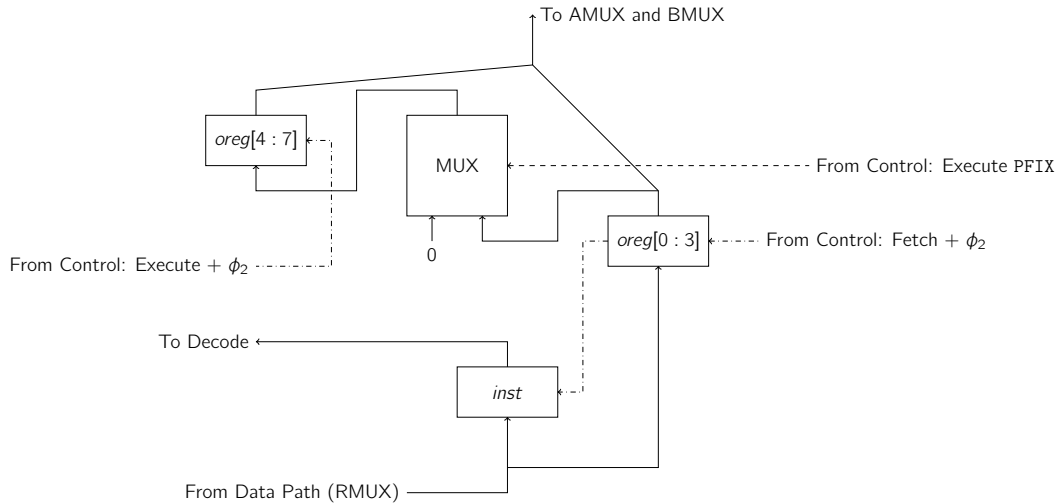
Hex 8: Update to inst/oreg registers



Hex 8: Update to inst/oreg registers



Hex 8: Update to inst/oreg registers



Hex 8: inst/oreg notes

- ▶ Each instruction fetch overwrites *inst* with opcode and *oreg*[0 : 3] with operand.
- ▶ The MUX is used to select the value to be stored in *oreg*[4 : 7].
- ▶ By default, *oreg*[4 : 7] set to zero during every Execute signal on ϕ_2
- ▶ The Control PFIX signal will instead select *oreg*[0 : 3] to be stored in *oreg*[4 : 7] during Execute.

Hex 8 ISA in full

Hex 8: ISA (1)

LDAM $areg \leftarrow mem[oreg]$: load from memory address *oreg* into *areg*

LDBM $breg \leftarrow mem[oreg]$: load from memory address *oreg* into *breg*

STAM $mem[oreg] \leftarrow areg$: store *areg* in memory address *oreg*

LDAC $areg \leftarrow oreg$: load constant into *areg*

LDBC $breg \leftarrow oreg$: load constant into *breg*

LDAP $areg \leftarrow pc + oreg$: load address in program

LDAI $areg \leftarrow mem[areg + oreg]$: load from memory into *areg*

LDBI $breg \leftarrow mem[breg + oreg]$: load from memory into *breg*

STAI $mem[breg + oreg] \leftarrow areg$: store *areg* in memory

Hex 8: ISA (2)

- BR** $pc \leftarrow pc + oreg$: branch relative unconditional
- BRZ** if $areg = 0$ then $pc \leftarrow pc + oreg$: branch relative zero
- BRN** if $areg < 0$ then $pc \leftarrow pc + oreg$: branch relative negative
- BRB** $pc \leftarrow breg$: branch absolute
- ADD** $areg \leftarrow areg + breg$: add registers and store in $areg$
- SUB** $areg \leftarrow areg - breg$: subtract registers and store in $areg$
- PFIX** concatenates 4-bit immediate operand with the 4-bit operand of the *next* instruction

Further reading

- ▶ Buses: [https://en.wikipedia.org/wiki/Bus_\(computing\)](https://en.wikipedia.org/wiki/Bus_(computing))
- ▶ <https://www.eolss.net/Sample-Chapters/C15/E6-45-02-07.pdf>
- ▶ Synchronous bus: <http://www.mathcs.emory.edu/~cheung/Courses/355/Syllabus/5-bus/synch-bus.html>
- ▶ Asynchronous bus: <http://www.mathcs.emory.edu/~cheung/Courses/355/Syllabus/5-bus/asynch-bus.html>
- ▶ Ethernet: <https://www.ccontrols.com/pdf/ExtV1N3.pdf>